

Java Streams

"Nothing runs until a terminal operation is called."

Recap: Lambdas

@FunctionalInterface

```
@FunctionalInterface
interface Calculator {
    int operate(int a, int b);
}

Calculator add      = (a, b) -> a + b;
Calculator multiply = (a, b) -> a * b;
Calculator max      = (a, b) -> a > b ? a : b;
```

Built-in functional interfaces

```
Function<String,Integer>
    .apply() -> s -> s.length()
```

```
Predicate<Integer>
    .test() -> n -> n % 2 == 0
```

```
Consumer<String>
    .accept() -> s -> println(s)
```

```
Supplier<Double>
    .get() -> () -> Math.random()
```

Composition:

```
Predicate.and()      isPositive.and(isEven) -> isPositiveAndEven
```

```
Predicate.negate()   isPositive.negate()   -> isNegative
```

```
Function.andThen()    doubler.andThen(addOne) -> x*2 then +1 (apply(5) = 11)
```

Recap: Lambdas

@FunctionalInterface

```
@FunctionalInterface
interface Calculator {
    int operate(int a, int b);
}

Calculator add      = (a, b) -> a + b;
Calculator multiply = (a, b) -> a * b;
Calculator max      = (a, b) -> a > b ? a : b;
```

Built-in functional interfaces

```
Function<String,Integer>
    .apply() -> s -> s.length()
```

```
Predicate<Integer>
    .test() -> n -> n % 2 == 0
```

```
Consumer<String>
    .accept() -> s -> println(s)
```

```
Supplier<Double>
    .get() -> () -> Math.random()
```

Composition:

```
Predicate.and()      isPositive.and(isEven) -> isPositiveAndEven
```

```
Predicate.negate()   isPositive.negate()   -> isNegative
```

```
Function.andThen()    doubler.andThen(addOne) -> x*2 then +1 (apply(5) = 11)
```

What is a Stream?

A stream is **not a data structure**; it describes a pipeline of operations over a source. Elements are pulled on-demand only when a terminal operation triggers execution.

Lazy

nothing runs until terminal

Single-use

consumed once only

Non-mutating

source is untouched

Source

collection / array
Stream.of / iterate



Intermediate

filter · map · sorted
limit · skip · peek



Terminal

collect · reduce
forEach · count

```
list.stream().filter(x -> x > 5).map(x -> x * 2).collect(Collectors.toList());
```

Stream Sources

`collection.stream()` Any Collection (List, Set, Deque...)

`collection.parallelStream()` Parallel execution across threads

`Arrays.stream(array)` From a T[] or primitive array

`Stream.of(a, b, c)` From explicit values

`Stream.empty()` Zero elements

`Stream.iterate(seed, f)` Infinite: 0, 1, 2, 3... (lazy)

`Stream.generate(supplier)` Infinite: Math::random, etc.

`IntStream.range(0, 10)` Primitive 0-9 (exclusive end)

`IntStream.rangeClosed(1, 10)` Primitive 1-10 (inclusive)

`Files.lines(path)` One stream element per line of a file

`str.chars()` IntStream of char codepoints

`Pattern.splitAsStream(str)` Regex-split tokens as a stream

Intermediate Operations - lazy, return a Stream

Filtering & Slicing

`filter(Predicate<T>)` Keep elements matching condition

`distinct()` Remove duplicates (uses equals)

`limit(long n)` Keep only first n elements

`skip(long n)` Discard first n elements

`takeWhile(Predicate)` J9+ Take while true, stop on first fail

`dropWhile(Predicate)` J9+ Drop while true, keep rest

Mapping & Transforming

`map(Function<T,R>)` $T \rightarrow R$ transform

`flatMap(Function<T,Stream<R>>)` flatten nested streams

`mapToInt(ToIntFunction)` $\rightarrow$ IntStream (no boxing)

`mapToLong / mapToDouble` $\rightarrow$ primitive streams

`mapMulti(BiConsumer)` J16+ 1-to-many flexible mapping

`flatMapToInt/Long/Double` flatten into primitive stream

Ordering & Peeking

`sorted()` Natural order

`sorted(Comparator)` Custom order

`peek(Consumer)` Debug — inspect without modifying

Terminal Operations - trigger execution, consume the stream

<code>collect(Collector)</code>	<code>R</code>	Gather into list, map, set, string...	<code>reduce(BinaryOp)</code>	<code>Optional<T></code>	Fold all elements into one value
<code>toList()</code>	<code>List<T></code>	Shorthand; unmodifiable (Java 16+)	<code>reduce(id, BinaryOp)</code>	<code>T</code>	Same, with identity starting value
<code>toArray()</code>	<code>Object[]</code>	Convert to array	<code>findFirst()</code>	<code>Optional<T></code>	First element — deterministic
<code>forEach(Consumer)</code>	<code>void</code>	Side-effect action on each element	<code>findAny()</code>	<code>Optional<T></code>	Any element — faster in parallel
<code>forEachOrdered(Consumer)</code>	<code>void</code>	forEach respecting encounter order	<code>anyMatch(Predicate)</code>	<code>boolean</code>	True if at least one matches
<code>count()</code>	<code>long</code>	Number of elements in stream	<code>allMatch(Predicate)</code>	<code>boolean</code>	True if every element matches
<code>min(Comparator)</code>	<code>Optional<T></code>	Smallest by comparator	<code>noneMatch(Predicate)</code>	<code>boolean</code>	True if no element matches
<code>max(Comparator)</code>	<code>Optional<T></code>	Largest by comparator	= short-circuit (stops early, works on infinite streams)		

Collectors - what you pass to collect()

Into collections

```
toList() · toSet()
```

```
toUnmodifiableList() · toUnmodifiableSet()
```

```
toCollection(ArrayList::new)
```

Strings & math

```
joining() · joining(", ") · joining(", ", "[" , "]")
```

```
counting() · summingInt/Long/Double()
```

```
averagingInt() · summarizingInt() · reducing()
```

Into maps

```
toMap(keyFn, valueFn)
```

```
toMap(keyFn, valueFn, mergeFunction)
```

```
toUnmodifiableMap(...)
```

Grouping & partitioning

```
groupingBy(classifier) → Map<K, List<T>>
```

```
groupingBy(classifier, downstream)
```

```
partitioningBy(predicate) → Map<Boolean, List<T>>
```

groupingBy example

```
record Person(String name, String dept){}
var people = List.of(
    new Person("Alice", "Eng"),
    new Person("Bob", "Eng"),
    new Person("Carol", "HR"));
```

```
Map<String, List<Person>> byDept =
    people.stream()
        .collect(
            groupingBy(Person::dept));
// {Eng=[Alice,Bob], HR=[Carol]}
```

Primitive Streams - IntStream / LongStream / DoubleStream

Avoid boxing overhead and get extra math-friendly methods:

```
IntStream.range(1, 6).sum()
```

Method	Returns	Description
<code>sum()</code>	<code>int / long / double</code>	Sum of all elements
<code>average()</code>	<code>OptionalDouble</code>	Average of elements
<code>summaryStatistics()</code>	<code>IntSummaryStatistics</code>	min, max, sum, avg, count at once
<code>min()</code> / <code>max()</code>	<code>OptionalInt/Long/Double</code>	Without needing a Comparator
<code>boxed()</code>	<code>Stream<Integer></code>	Convert back to boxed stream
<code>asLongStream()</code>	<code>LongStream</code>	Widen Int → Long
<code>asDoubleStream()</code>	<code>DoubleStream</code>	Widen Int/Long → Double
<code>range(0, 10)</code>	<code>IntStream</code>	Integers 0-9 (exclusive end)
<code>rangeClosed(1, 10)</code>	<code>IntStream</code>	Integers 1-10 (inclusive)
<code>mapToObj(IntFunction)</code>	<code>Stream<T></code>	Convert to object stream

Cheat Sheet

Create	Intermediate (lazy)	Terminal (eager)
<code>collection.stream()</code>	<code>filter(Predicate)</code>	<code>collect(Collector)</code>
<code>Arrays.stream(arr)</code>	<code>map(Function<T,R>)</code>	<code>forEach(Consumer)</code>
<code>Stream.of(a,b,c)</code>	<code>flatMap(fn)</code>	<code>reduce(id, BinaryOp)</code>
<code>Stream.iterate(0,n->n+1)</code>	<code>distinct() / sorted()</code>	<code>count() / min() / max()</code>
<code>Stream.generate(Supplier)</code>	<code>limit(n) / skip(n)</code>	<code>findFirst() / findAny()</code>
<code>IntStream.range(0,10)</code>	<code>peek(Consumer) ← debug</code>	<code>anyMatch / allMatch / noneMatch</code>

```
names.stream().filter(n ->
n.startsWith("A")).map(String::toUpperCase).sorted().limit(3).collect(Collectors.toUnmodifiableList());
```